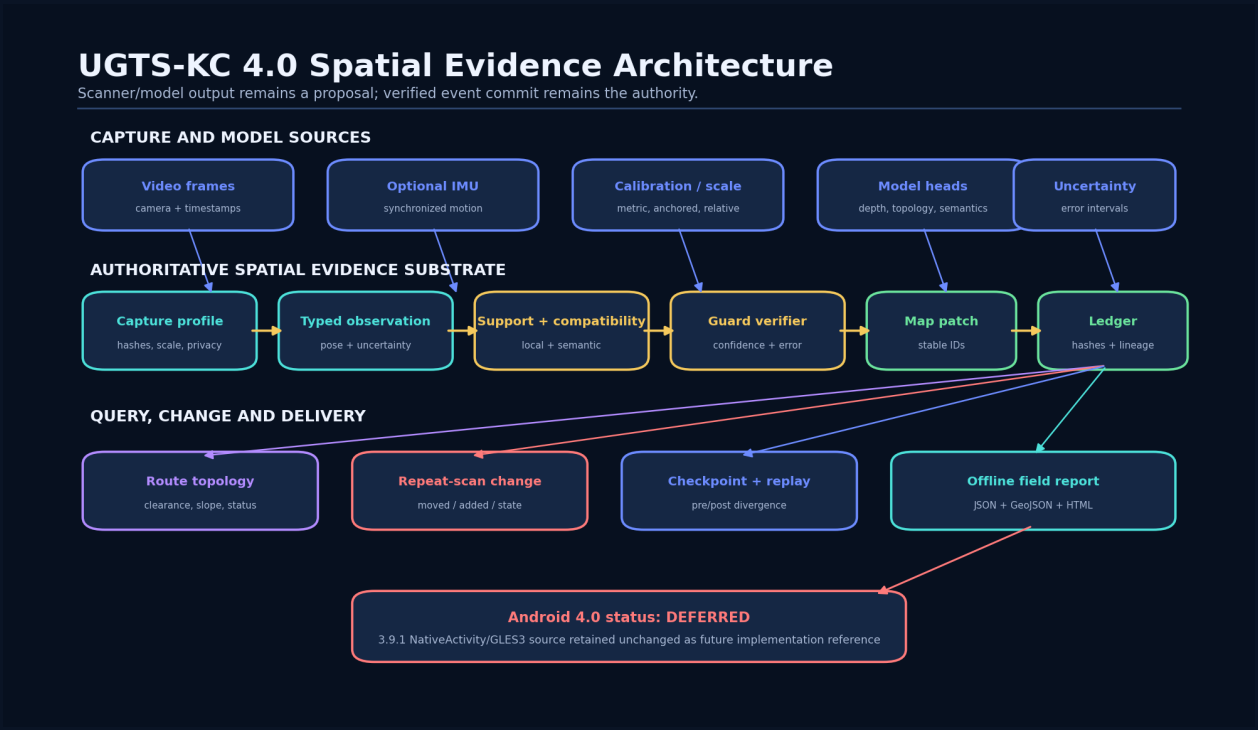
UGTS-KC 4.0

SPATIAL EVIDENCE LEDGER

Spatial evidence, route topology, uncertainty, change lineage and offline field reports



Prepared for	Tom Klootwijk
Requester-supplied identifier	NL200678942
Requester-supplied date of birth	10 July 1990
Version	4.0.0
Release status	Spatial substrate delivered; Android native application intentionally deferred

Requester attribution and identifiers are preserved as supplied and are not independently verified. This is a technical package, not legal proof of identity, authorship, ownership, priority or patentability.

Document control and reading rule

Release	UGTS-KC 4.0.0 - Spatial Evidence Ledger
Project schema	`ugts-kc-spatial-project-4.0`
New namespace	`src/ugts_kc4/`
Mechanism delta	M450–M509 (60 additions; combined range M001–M509)
Automated tests	380 passing: 276 retained + 104 new
Reference vertical slice	SafeRoute + DamageDelta, 17 verified events, four repeat-scan changes, hash-checked replay
Android status	Deferred; attached 3.9.1 working sources retained unchanged as the future implementation base

EVIDENCE BOUNDARY

The authoritative object is not a video frame, point cloud, render mesh or neural-network output. Those are evidence sources or downstream projections. A discrete map fact exists in the 4.0 reference runtime only after support, compatibility, relation/guard, confidence, uncertainty, scale and provenance gates accept a proposal and the ledger commits an atomic patch with lineage.

Contents

1. Executive upgrade decision

UGTS-KC 4.0 completes the substrate expansion required before a phone-specific native scanner is attempted. It extends the retained 3.9.1 implementation with a typed spatial-evidence model, conservative uncertainty, stable map identity, accessible route topology, repeat-scan change detection, content-addressed proposal deltas, checkpoint/replay and self-contained field reporting.

DECISION

GO for UGTS-KC 4.0 as an additive spatial evidence and topological mapping substrate. POSTPONE the new Android native capture/viewer application until the 4.0 record, verifier, ledger, route, change and export interfaces are frozen. Retain the supplied 3.9.1 NativeActivity/C++/EGL/OpenGL ES implementation unchanged and use it as the implementation reference in the later device phase.

1.1 Concrete deliverables

Deliverable	Location / content	Status
Technical release report	`report/UGTS_KC_4_0_Spatial_Evidence_Ledger.pdf`	Delivered
Reference runtime	`src/ugts_kc4/`: records, supports, keys, verifier, topology, ledger, change, export, CLI	Tested
Machine-readable contract	JSON Schema, 60-row catalog, formal contracts and source basis under `spec/`	Delivered
Vertical slice	Editable SafeRoute + DamageDelta project, baseline/changed ledgers, GeoJSON and offline HTML	Executed
Validation evidence	Full test capture, schema/catalog/demo checks, source-freeze checks and hashes under `validation/`	PASS
Android implementation	New APK/AAB, capture pipeline, model deployment and named-phone optimization	Deferred
Frozen Android reference	Supplied 3.9.1 project/template and exporters with unchanged tree/file hashes	PASS

1.2 Canonical shorthand

```
video / still frames / optional IMU / declared scale anchor
-> capture profile + typed observations + uncertainty
-> local support + semantic/topological compatibility
-> relation interval + guard class
-> confidence + numeric error + metric scale + provenance gates
-> verified proposal
-> deterministic conflict policy + atomic map patch
-> stable nodes/edges + evidence + lineage + pre/post hashes
-> route queries + repeat-scan changes + checkpoint/replay
-> canonical JSON / GeoJSON / self-contained offline HTML
```

2. Source continuity and evidence discipline

The 4.0 design is derived from, and remains compatible with, the supplied project line. The source packages establish a consistent authority chain while contributing different implementation layers: the 2.0 pattern/kinematic substrate, the 3.0 stable scene and replay shell, the 3.6 content-addressed definition graph, the 3.6.2 SCLP support/indexing profile, the 3.9 deterministic/offline deployment workflow, and the GPU-native precision and claims boundaries.

2.1 Immediate source basis

Source	Role in 4.0	SHA-256 prefix
UGTS-KC 2.0	Pattern, field, topology, SE(3), dynamics and canonical event calculus.	`a29dd9959ebd300c...`
UGTS-KC Two Hands 3.0	Stable scene identity, geometry error contracts, BVH, event commit, replay and interchange.	`183ffa4e2bf29cfa...`
UGTS-KC 3.6	Content-addressed definitions, explicit operation order and referential verification.	`e21433cd8378368d...`
UGTS-KC 3.6.2 SCLP	Cone/sphere support, log-polar metric, uncertainty intervals and exact 64-bit layouts.	`6987490a0a7b7ed7...`
UGTS-KC 3.9	Versioned projects, deterministic runtime and self-contained offline HTML delivery.	`d7ee3c91abf60e35...`
UGTS-GN 1.1	Normalized baseline, support/compatibility/guard order, precision and performance boundaries.	`3a6b6c57118ca4e2...`
Attached 3.9.1 package - compact	Working 3.9.1 Python/mobile/native Android source baseline.	`15eb99dd3ca368d5...`
Attached 3.9.1 package - extended	Same package plus additional extracted Android source/build material and retained report.	`1833255f0acadb74...`

2.2 Inherited architectural commitments

- **Query-first authority.** Support, compatibility, guard, verified event, transition and lineage remain in that order.
- **Coordinates are not identity.** Stable node/edge IDs, route class, evidence, revision and lineage remain separate from position.
- **Derived geometry is bounded.** A visual mesh or plan view never becomes measurement or collision authority without a separate error contract.
- **Definitions and model versions are inspectable.** Calibration, model-head versions, source hashes and definition hashes are explicit records rather than hidden application state.
- **Finite packing remains finite.** A 64-bit key is an address accelerator, not complete state, infinite detail or a substitute for uncertainty.
- **Deployment can be offline.** Canonical project and ledger records can produce a self-contained HTML field report without a CDN or network asset.

EVIDENCE BOUNDARY

The 4.0 mechanisms are engineering additions. They are not silently attributed to earlier source documents, and the attached source claims are not expanded into SLAM accuracy, medical validity, structural-safety certification, emergency-navigation guarantees or phone performance without independent measurement.

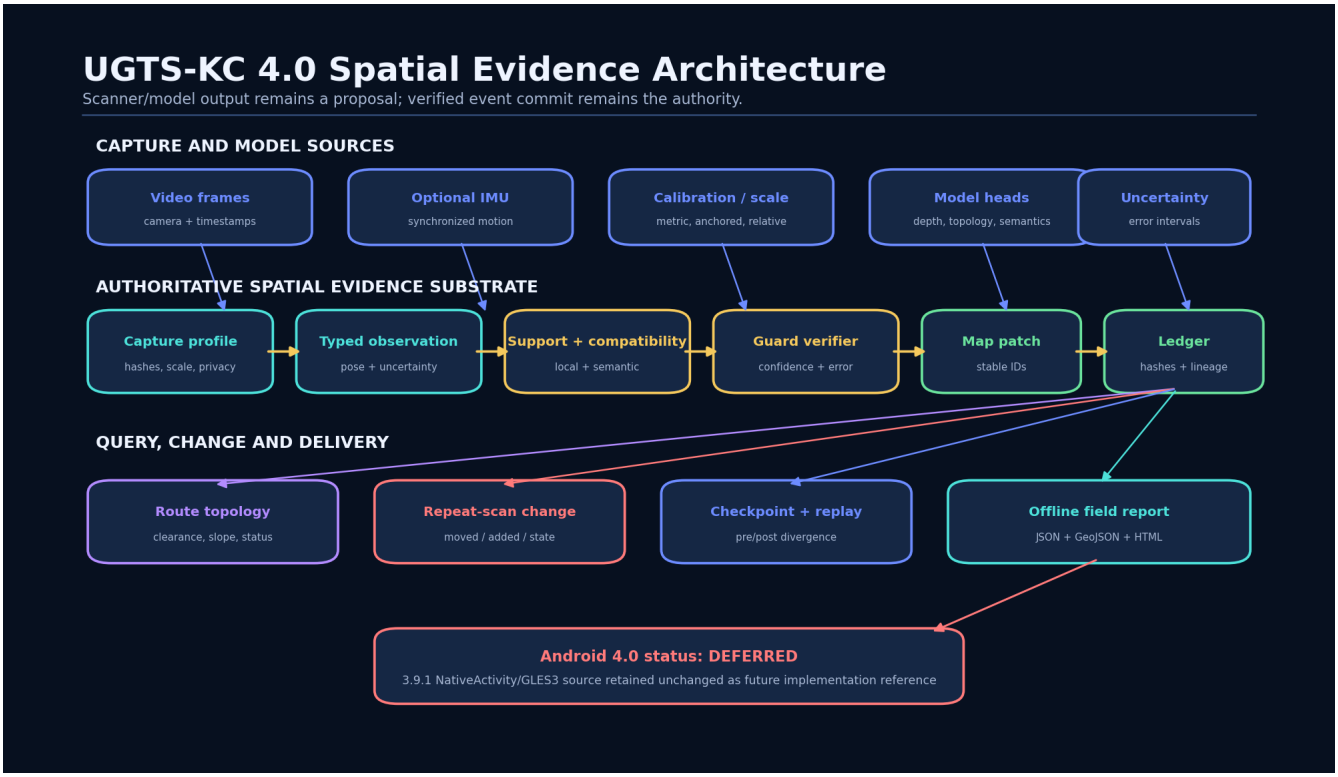
3. Canonical 4.0 architecture

3.1 Formal release definition

The retained substrate is extended by a spatial evidence layer:

$$\mathrm{KC4} = \mathrm{KC3.9.1} + (C, O, U, Q, V, M, R, L, \Delta, X, G)$$

where C is capture/calibration, O typed observations, U uncertainty and relation intervals, Q support and spatial keys, V verification, M stable map state, R route topology, L ledger/checkpoints/replay, Δ change/merge, X interchange/export, and G governance and evidence boundaries.



3.2 Single source of spatial truth

A `SpatialProject` holds capture profiles, support volumes, compatibility and verification policies, route policies, change policy, initial map and export settings. A `SpatialLedger` holds the authoritative initial map, ordered committed events and optional checkpoints. The same canonical dictionaries are used for validation, content hashing, replay, CLI queries and offline export.

3.3 Public reference modules

Module	Primary responsibility
`canonical.py`	Canonical JSON, deterministic SHA-256 content hashes and numeric normalization.
`model.py`	Intervals, poses, capture profiles, observations, map nodes/edges, patches and map state.
`support.py`	Sphere, AABB and finite cone-frustum support plus semantic compatibility policies.
`index.py`	Two exact 64-bit key profiles and deterministic spatial hash index.
`verify.py`	Guard intervals, verification policies, event proposals and reason-coded decisions.
`topology.py`	Route edge admission, deterministic shortest path, components and topology validation.
`ledger.py`	Ordered commits, conflicts, events, pre/post hashes, checkpoints and replay.
`change.py`	Repeat-scan moved/state/metric/add/remove candidates and multi-source delta merge.
`project.py`	Versioned project model, cross-reference validation, load/save and content hash.
`export.py`	GeoJSON and self-contained offline HTML report.
`templates.py` / `cli.py`	Complete reference project/demo generation and command-line workflow.

4. Capture, calibration and model provenance

4.1 Capture profile contract

A capture profile binds the acquisition semantics needed to interpret observations:

$$C = (\text{device}, \text{camera}, h_{cal}, F, t_u, \text{models}, s, \text{privacy}).$$

The reference record includes a stable profile ID, device family, camera model, calibration hash, scale mode, units per meter, coordinate frame, timestamp unit, model-version map, privacy policy and optional scale-anchor ID.

Field	Allowed/reference values	Authority role
Scale mode	`metric_calibrated`, `anchored`, `relative`, `unknown`	Metric route/clearance proposals may require the first two.
Calibration hash	Content digest or explicit `unverified`	Associates every measurement with the calibration that produced it.
Coordinate frame	Handedness, up axis and units	Prevents silent axis, unit or floor-plane changes.
Model versions	Depth, topology, semantics, uncertainty heads	Makes mixed-model evidence and model upgrades auditable.
Privacy policy	Local-only, redacted export, consented sync, unrestricted	Provides a declared gate before evidence leaves a device.
Scale anchor	Stable anchor ID	Separates a real scale witness from a model estimate.

4.2 Why the phone front end is not yet authoritative

A later mobile application may provide camera frames, timestamps, optional IMU and on-device model output. In 4.0 those are deliberately modeled as sources of `Observation` proposals. The substrate already defines what the future app must supply and how its evidence will be rejected or committed; it does not pretend that the camera, SLAM or transformer stack has already been implemented.

5. Typed observations and conservative uncertainty

5.1 Observation record

Each observation contains stable observation/profile/frame IDs; timestamp; kind; pose; uncertainty; confidence; relation value and numerical error; support and compatibility policy IDs; semantic tags; source model and source hash; optional definition hashes; dynamic class; and whether metric scale is required.

5.2 Position and relation intervals

The reference uncertainty bound is conservative:

$$u_p = \begin{cases} \text{max\_error}, & \text{when explicitly supplied,} \\ 3\sqrt{\sigma_x^2 + \sigma_y^2 + \sigma_z^2}, & \text{otherwise.} \end{cases}$$

A scalar relation observation g with numerical error e is treated as the interval

$$I_g = [g - e,\ g + e].$$

The guard classifier distinguishes interior, exterior, guard-band and crossing/ambiguous cases by comparing this interval with the declared event margin. The original continuous value and its uncertainty remain available; no one-bit class replaces them.

5.3 Stable evidence links

Accepted patches preserve evidence IDs and lineage labels. Re-observing a doorway or route edge adds evidence and may update state or geometry without changing the stable map identity. A rejected observation remains a reason-coded validation result and cannot mutate the map.

RELEASE RESULT

Uncertainty is operational rather than decorative. Route clearance uses a conservative lower bound; route slope uses a conservative upper bound; movement is verified only when the lower displacement bound exceeds the change threshold; and numeric error must remain within both policy and proposal event margins.

6. Support, compatibility and spatial indexing

6.1 Local support volumes

The reference runtime provides three finite supports:

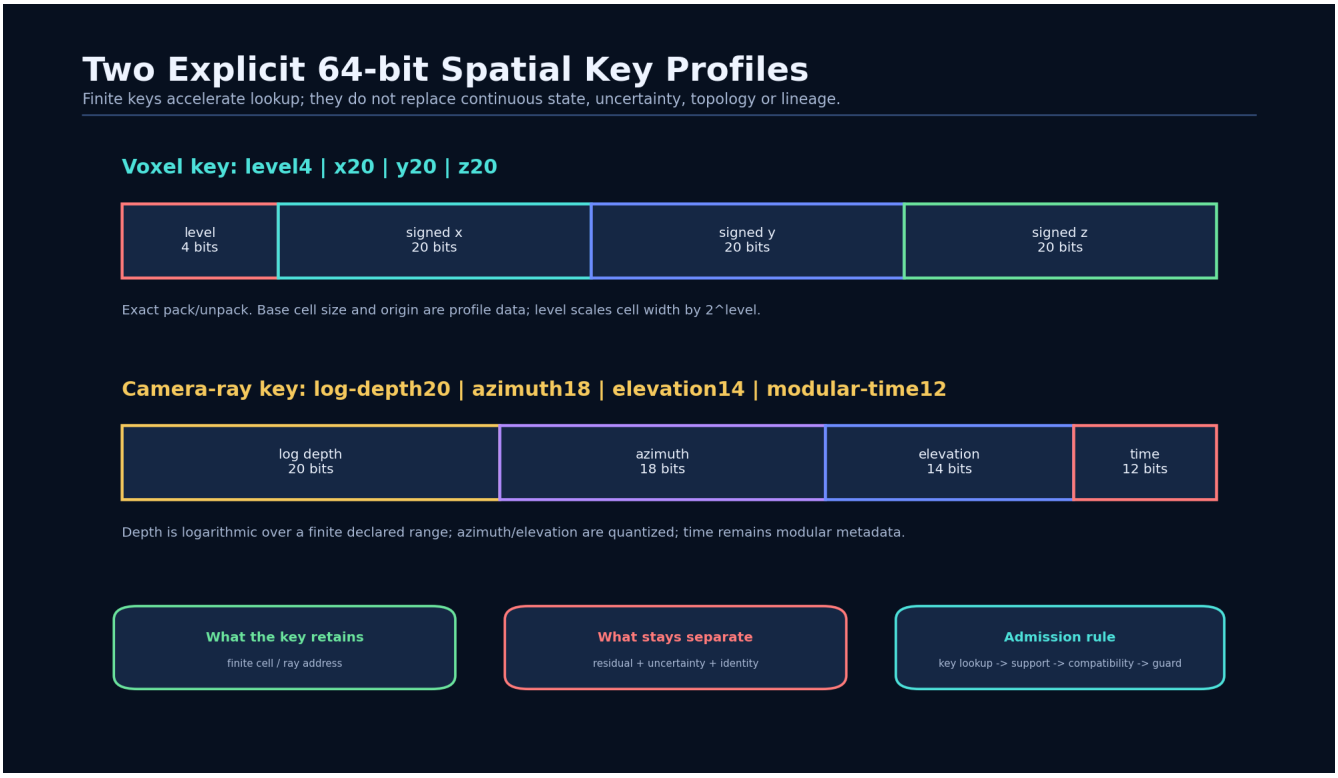
- **Sphere support**: radial admission with uncertainty expansion.
- **AABB support**: finite axis-aligned bounds with conservative uncertainty padding.
- **Cone-frustum support**: near/far distance and angular aperture around an origin/axis, suitable for camera or sensor relevance.

Support is a coarse local admission step. It does not establish semantic compatibility or a verified relation.

6.2 Compatibility policies

Compatibility may constrain observation kind, dynamic class, required and forbidden tags, floor/sheet labels, source model or application-specific semantic fields. This preserves the substrate rule that co-location is necessary but not sufficient for coupling.

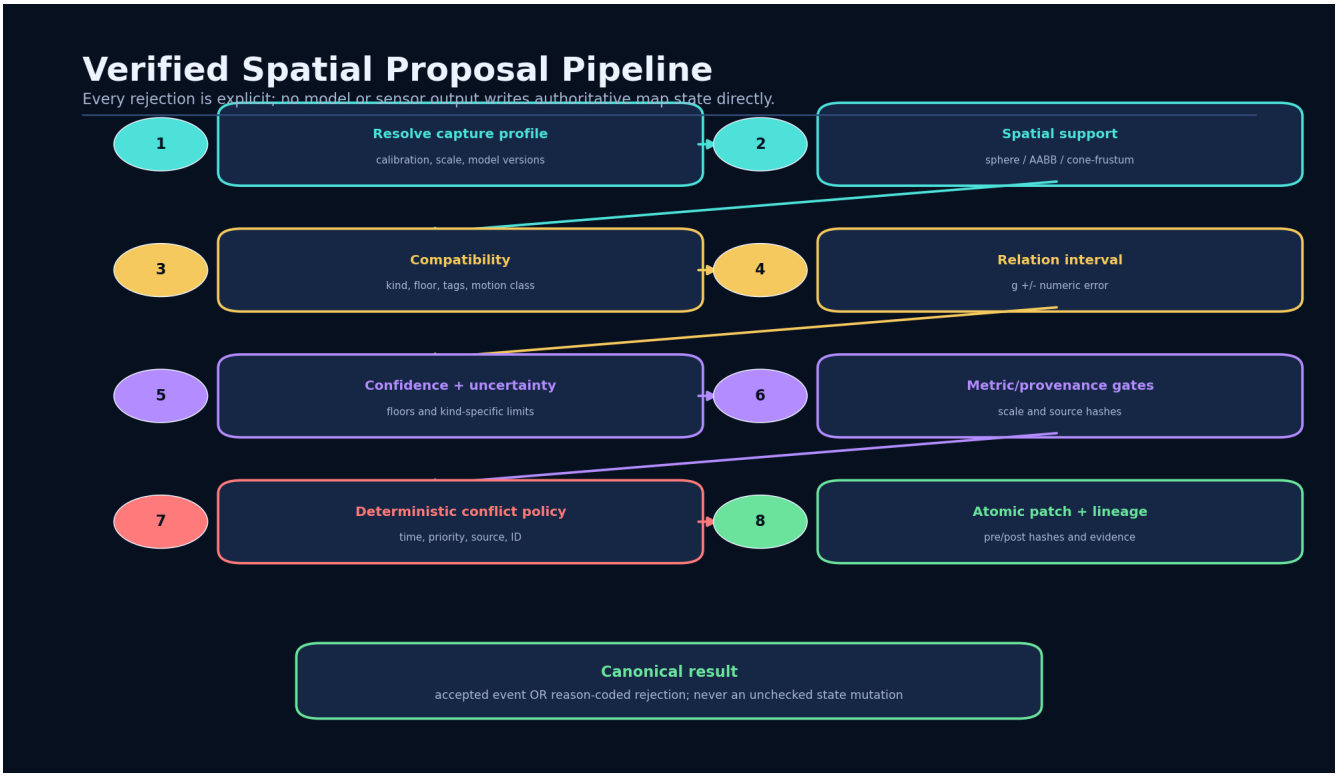
6.3 Two exact 64-bit key profiles



- The **voxel key** allocates 4 bits to level and 20 signed bits each to x, y and z cell coordinates. Base cell size and origin remain profile data.
- The **camera-ray key** allocates 20 bits to finite log-depth, 18 to azimuth, 14 to elevation and 12 to modular time.

- Both layouts have tested pack/unpack round trips. Neither stores residual, uncertainty, semantic identity, route class or lineage.
- The spatial hash returns deterministic candidate order, after which support, compatibility and guard verification still run.

7. Verified proposal and event commit



7.1 Verification predicate

A proposal is accepted only when all required gates hold:

$$\begin{aligned}\text{verified} = {} & \text{profile\_known} \wedge \text{support} \wedge \text{compatible} \\ & \wedge \text{guard\_status} \wedge \text{relation\_class} \\ & \wedge (c \geq c_{min}) \wedge (e \leq e_{max}) \wedge (e \leq m) \\ & \wedge (u_p \leq u_{max}(\text{kind})) \wedge \text{metric/provenance gates}.\end{aligned}$$

Every failed term emits an explicit reason such as `outside_support`, `metric_scale_required`, `confidence_below_floor`, `position_uncertainty_exceeds_limit` or `source_hash_required`.

7.2 Deterministic ordering and conflict policy

Accepted proposals are sorted by event time, descending priority, source and proposal ID. A patch may add, replace or remove a node/edge, update project metadata, attach evidence and append lineage. Incompatible same-time writes receive explicit rejections; they do not become last-writer-wins mutations hidden inside a device or model adapter.

7.3 Atomic patch identity

Each committed event records sequence, proposal, verification decision, patch, pre-state hash, post-state hash, evidence and lineage. The hash is a divergence and integrity check; full state, evidence and lineage remain authoritative.

8. Stable map identity and accessible route topology

8.1 Node and edge separation

A `MapNode` has stable ID, semantic kind, pose, uncertainty, state, metrics, evidence IDs, revision and lineage. A `MapEdge` has stable ID, source/target node IDs, edge kind, direction, state, metrics, evidence IDs, revision and lineage. Coordinates, object identity, route identity and evidence identity are deliberately distinct.

8.2 Route admission contract

A route policy declares minimum clearance, maximum slope, minimum confidence, maximum uncertainty, allowed statuses and whether unknown edges may be used with a penalty. For an edge:

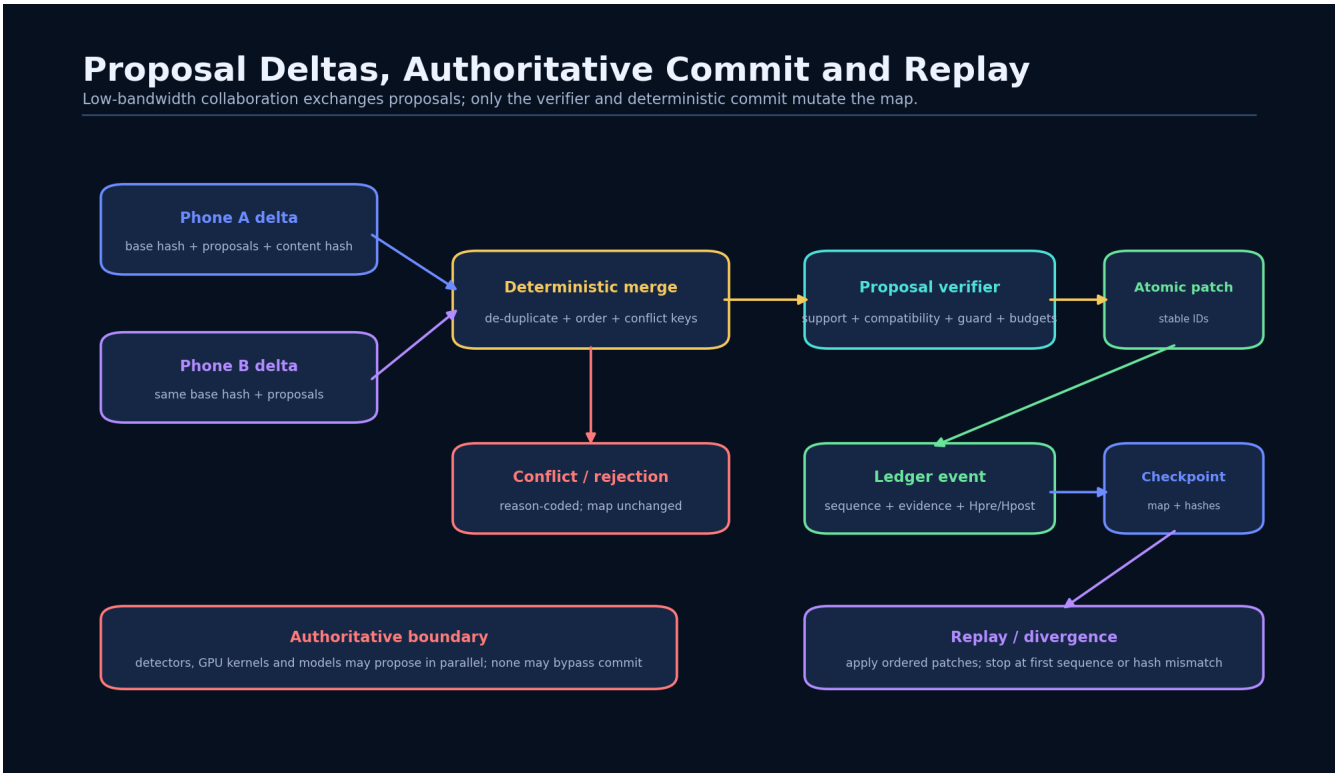
$$I_c = [c - e_c, c + e_c], \qquad I_s = [\max(0, s - e_s), s + e_s].$$

The edge is rejected when the lower clearance bound is below policy, the upper slope bound is above policy, confidence is too low, uncertainty is too high, status is blocked/closed or length is invalid. Admitted cost is length plus risk; deterministic Dijkstra tie-breaking uses stable edge/node signatures.

8.3 Safety interpretation

The route graph is a descriptive, policy-bound map. It can record passable, blocked, unknown and measured clearance/slope states, but 4.0 does not certify a building, floor or structure as safe. Emergency or accessibility use requires field validation, application governance and an independently maintained fallback process.

9. Ledger, checkpoint, replay and collaborative deltas



9.1 Checkpoint and replay

A checkpoint contains sequence, canonical map snapshot and snapshot hash. Replay begins from the initial state or a verified checkpoint, applies ordered events, checks every pre/post hash and stops at the first sequence or hash mismatch. The reference vertical slice replays to the identical final state hash.

9.2 Proposal deltas

A low-bandwidth delta contains base-state hash, source ID, proposals and a content hash. Multi-source merge first requires the same base state, de-duplicates identical proposals and sorts deterministically. Conflicting target/field writes are returned as explicit merge conflicts. Proposal exchange does not bypass verification or commit.

9.3 Lineage and evidence

The ledger preserves how a state was reached: source observation, accepted or rejected gate result, patch, sequence, prior state hash, resulting state hash and lineage label. A compact digest may accelerate comparison but never substitutes for the event stream.

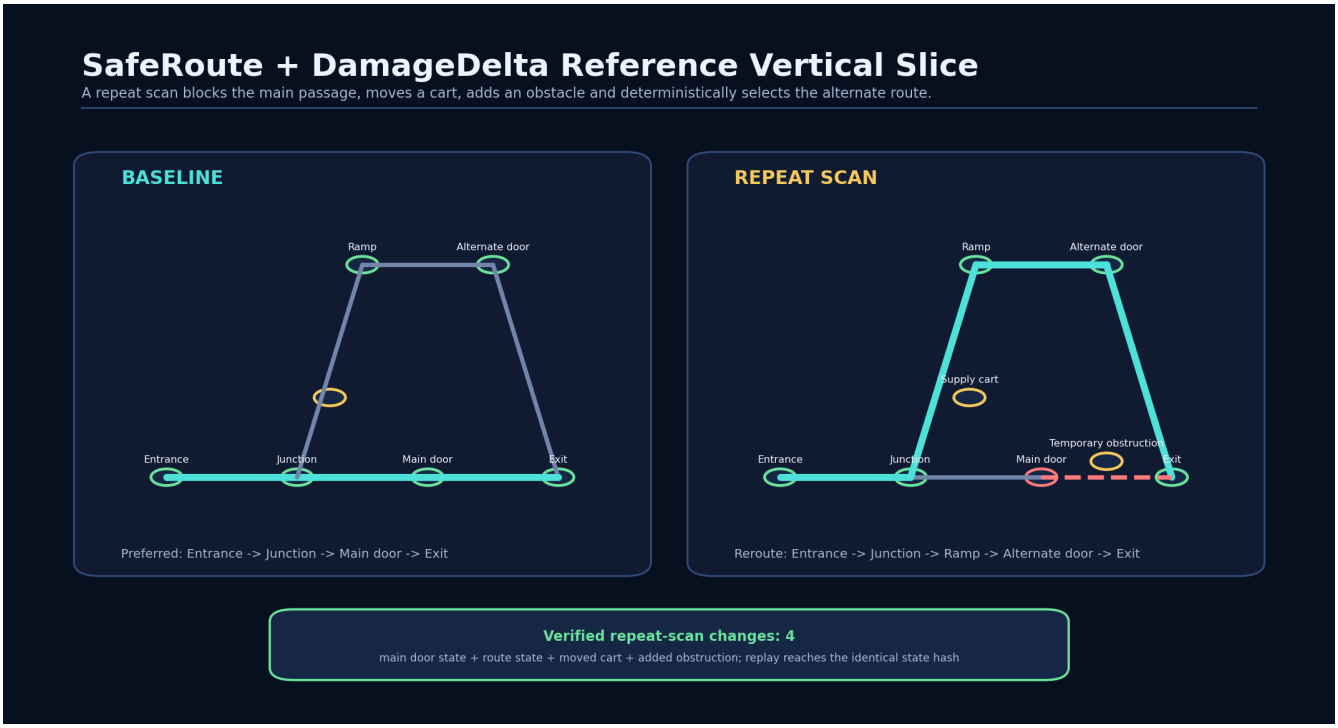
10. Repeat-scan change ledger and reference vertical slice

10.1 Change classes

The reference comparator uses stable IDs to produce moved, movement-ambiguous, selected-state-changed, metric-changed, added and removed candidates. For a node observed in both scans, displacement is bounded by

$$I_d = [\max(0, d - u_0 - u_1),\ d + u_0 + u_1].$$

Movement is verified only when the lower bound exceeds the declared threshold. When only the upper bound exceeds it, the result is explicitly ambiguous.



10.2 Captured demonstration result

Metric	Captured result
Baseline route	Entrance → Junction → Main door → Exit; cost 12.12
Repeat-scan route	Entrance → Junction → Ramp → Alternate door → Exit; cost 17.1043
Rejected route edge	`E-MAIN-EXIT: status_not_allowed:blocked`
Baseline / change events	13 / 4; total ledger events 17
Verified change candidates	Main route state, main door state, moved supply cart, added temporary obstruction
Project hash	`042b35beefc386222921faef1e4737ed3ad05ab341470617b3593a4bc05a41d4`
Event-stream hash	`c4b8012977c8282917ac0989790c36a514dd62a548971abca35e8de7e7a04a3a`
Final/replay state hash	`3ce4d3b61520429d3d99747f6907ecf6f36dc86a2a38b3c40480d8cb61dc7187`
Replay result	**PASS**

11. Project, export and offline field delivery

11.1 Versioned project record

The editable `project.json` contains schema/version metadata, capture profiles, support/compatibility/verification policies, route policy, change policy, initial map and export settings. Cross-reference validation catches unknown policy/profile/support IDs, bad route endpoints and topology issues before execution.

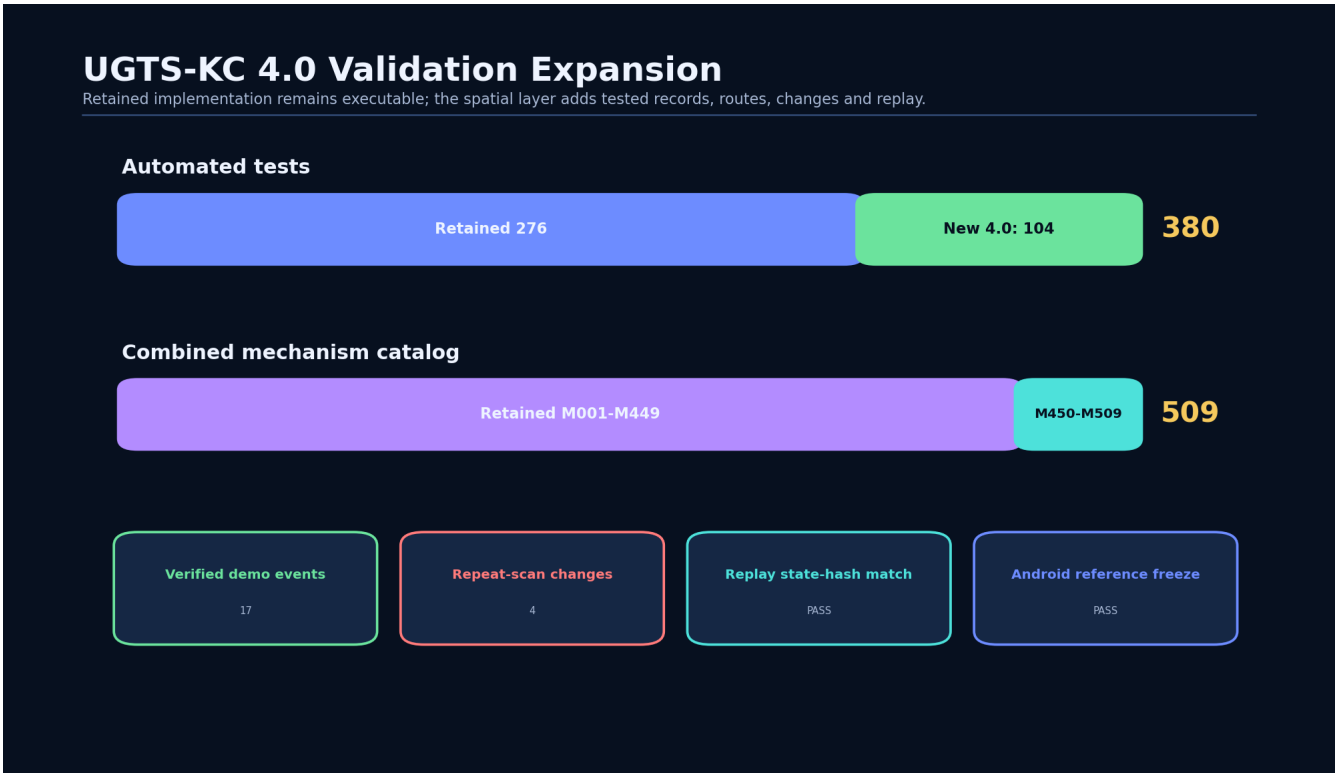
11.2 Canonical outputs

Output	Purpose	Authority
Spatial project JSON	Definitions, policies and initial map	Authoritative configuration
Ledger JSON	Initial map, ordered events and checkpoints	Authoritative event/state record
GeoJSON	Nodes, route lines and selected properties	Non-authoritative interchange projection
Offline HTML	Embedded plan SVG, route/change summaries, project/ledger data and save function	Non-authoritative field report
Optional future mesh/glTF	Visual inspection or device viewer	Derived unless separately promoted

The generated HTML contains no CDN, web font, package manager or network asset. It can be transferred and opened as a local file.

11.3 Command-line workflow

```
PYTHONPATH=src python -m ugts_kc4 info
PYTHONPATH=src python -m ugts_kc4 demo build/spatial_demo --json
PYTHONPATH=src python -m ugts_kc4 validate \
  examples/safe_route_spatial_ledger/project.json --json
PYTHONPATH=src python -m ugts_kc4 route \
  examples/safe_route_spatial_ledger/project.json \
  examples/safe_route_spatial_ledger/changed_ledger.json \
  N-ENTRANCE N-EXIT --policy wheelchair-reference --json
PYTHONPATH=src python -m ugts_kc4 android-status
```



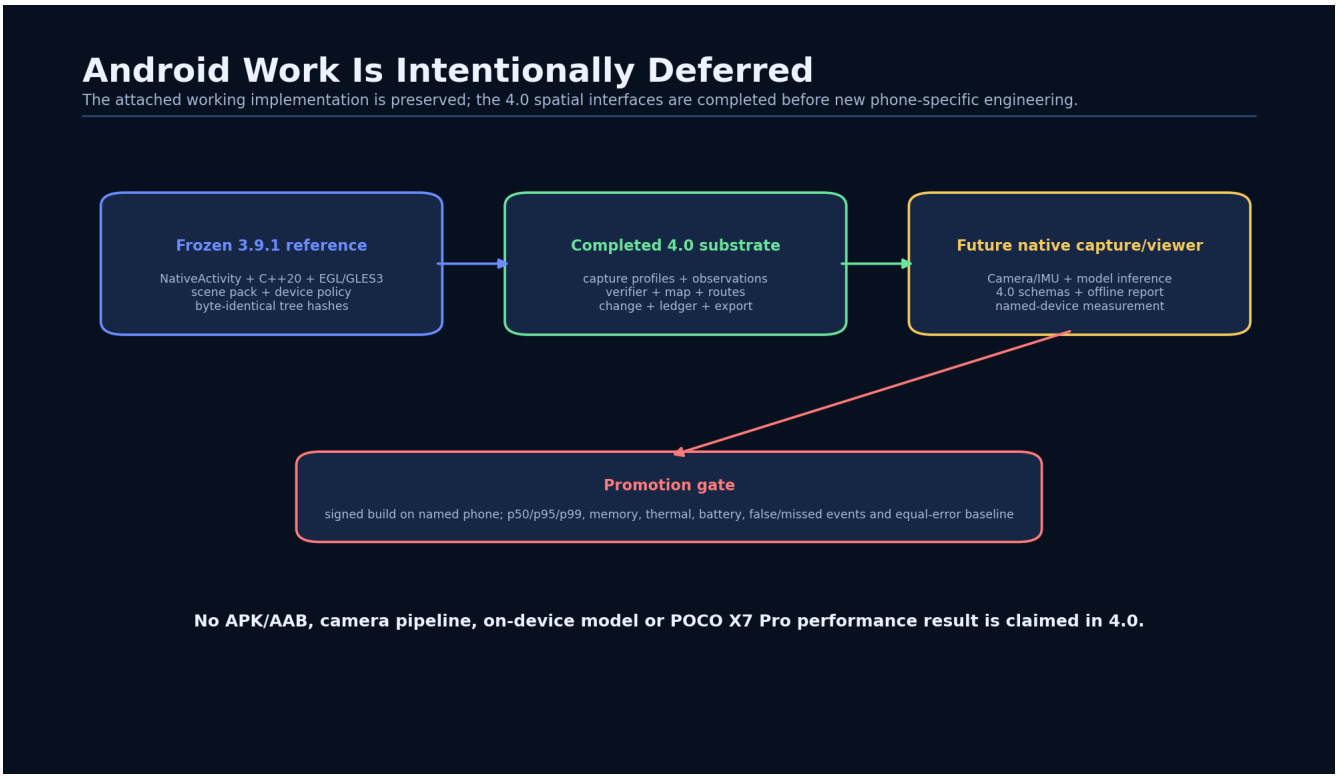
12. Validation and release evidence

Gate	Result	What is established
Python compile	**PASS**	All retained and new source modules compile with `compileall`.
Automated tests	380/380 PASS	276 retained + 104 new tests across capture, support, keys, verification, maps, routes, ledger, change, merge and export.
Schema validation	**PASS**	Example project validates with JSON Schema Draft 2020-12.
Catalog validation	**PASS**	M450–M509 is contiguous; combined range reaches M509.
Demo execution	**PASS**	17 commits, deterministic reroute, four verified changes and matching replay hash.
Offline export	**PASS**	HTML is self-contained; GeoJSON/project/ledger data parse and hash as expected.
Android freeze	**PASS**	Retained 3.9.1 Android project/template/exporter hashes match the supplied baseline.
Native phone build	**DEFERRED**	No 4.0 APK/AAB or named-device benchmark is claimed.

RELEASE RESULT

The executable evidence establishes internal consistency of the bundled reference implementation. It does not establish monocular-SLAM accuracy, calibration quality on a physical camera, cross-device floating-point identity, field-navigation safety, structural or clinical validity, or performance on the requester's phone.

13. Android native implementation postponement



13.1 Frozen working reference

The supplied 3.9.1 native/mobile implementation is preserved without modification:

Reference	Release hash	Status
`android/UGTSKC391Signature`	`0a6e46c8bc0cd73f65bffed0b4467f99297b37644db0655bf5d547edf110a43`	PASS
`src/ugts_kc3/android_template`	`57817e5c91f9d5307b1a53b5f2d0be2260f23ff741621246bf5abbd7933da5`	PASS
`src/ugts_kc3/mobile3d.py`	`b726be5c391d070b7e321c6e49657c3714255caec3a9a0c6bacb498966e48a8`	PASS
`src/ugts_kc3/androidexport.py`	`db97c1717a9187e5b9f49257d7e1eb7891e290b0bd0927b3bac3cd277cdbb`	PASS

13.2 Required future implementation path

The later phone-specific application must begin with the retained NativeActivity/C++20/EGL/OpenGL ES source and bind the frozen 4.0 interfaces rather than inventing a separate state model. The sequence is:

1. camera/IMU acquisition and device calibration;
2. keyframe tracking and model inference adapter;
3. construction of 4.0 capture profiles and observations;
4. verifier and ledger integration;
5. route/change queries and offline report generation;
6. signed native build for the named phone;
7. device tests for camera behavior, memory, p50/p95/p99 latency, thermal state, battery, false/missed events and equal-error comparison.

EVIDENCE BOUNDARY

Version 4.0 claims no new APK/AAB, Camera2/CameraX pipeline, IMU fusion, SLAM implementation, distilled transformer model, Vulkan inference backend or POCO X7 Pro performance result. These are explicit later-stage engineering tasks gated by the completed substrate and named-device measurement.

14. Application boundary and kill criteria

14.1 Numerical and evidence kill criteria

Simplify or reject the spatial layer when:

- calibration, coordinate-frame or scale identity is unavailable for a measurement-critical event;
- relation/numeric/quantization uncertainty exceeds the event margin or changes ordering;
- movement, clearance or slope intervals cannot classify the required threshold;
- source/model/definition hashes required by policy are missing or inconsistent;
- stable object association cannot distinguish movement from re-identification failure;
- route status or topology is stale, contradictory or insufficiently observed.

14.2 Performance and storage kill criteria

- voxel/ray-key construction plus support/compatibility costs more than a conventional index at equal error;
- event, branch, evidence or checkpoint density removes the expected storage advantage;
- replay or multi-source merge cannot remain deterministic under the selected profile;
- offline reports become too large for field transfer or contain privacy-prohibited evidence;
- a conventional mapping stack is simpler, faster or more accurate for the target workload.

14.3 Application non-claims

- No autonomous emergency-navigation guarantee or structural-safety verdict.
- No medical diagnosis, orthotic/prosthetic manufacturing authority or clinical validation.
- No claim that image-only crack, leak, damage or accessibility classification is sufficient without appropriate sensors and professional review.
- No legal chain of custody, digital signature, ownership proof or privacy certification from content hashes alone.
- No claim that a 64-bit index stores complete geometry, uncertainty, topology or history.

15. Final upgraded definition and package map

FINAL DEFINITION

UGTS-KC 4.0 is a finite, content-addressed and replayable spatial evidence substrate. It binds acquisition to explicit capture/calibration profiles; represents model and sensor outputs as typed observations with conservative uncertainty; prunes them through finite spatial support and semantic compatibility; classifies relation intervals under declared guard margins; commits only verified deterministic patches; preserves stable node/edge identity, evidence and lineage; evaluates policy-bound routes; detects repeat-scan change by stable identity and bounded intervals; and exports canonical JSON, GeoJSON and self-contained offline reports. Android capture and phone-specific execution remain downstream implementation targets, not implicit features of the substrate.

15.1 Package map

Path	Contents
`src/ugts_kc4/`	4.0 spatial reference runtime and CLI.
`examples/safe_route_spatial_ledger/`	Complete editable project, baseline/changed ledgers, routes, changes, GeoJSON and offline report.
`spec/`	4.0 JSON Schema, formal contracts, source basis and catalogs through M509.
`docs/`	Getting started, API, evidence boundary, migration, release notes and Android deferral.
`report/`	This PDF, editable LaTeX source and retained legacy reports.
`android/`	Frozen 3.9.1 NativeActivity project used as future reference.
`tests/`	380-test retained-plus-new suite.
`validation/`	Test captures, schema/catalog/demo checks, manifests, hashes and PDF preflight evidence.
`manifest/` / `checksums/`	Final package tree, file manifest and SHA-256 checksums.

A. Mechanism catalog M450–M509

The 4.0 delta contains 60 engineering mechanisms. Full formulas, definitions, integration notes and validation states are supplied in CSV and JSON under `spec/`. The table below is the complete human-readable index.

A.1 Domain summary

Range	Domain	Count	Representative mechanisms
M450–M457	Capture and calibration	8	Versioned capture profile, Camera calibration content digest, Coordinate-frame declaration...
M458–M467	Observation and uncertainty	10	Typed observation proposal, Normalized pose record, Three-axis uncertainty record...
M468–M475	Spatial support and indexing	8	Spherical support volume, Axis-aligned box support, Finite cone-frustum support...
M476–M483	Map and identity	8	Stable map-node identity, Node revision lineage, Stable map-edge identity...
M484–M491	Topology and route	8	Room/portal/waypoint node classes, Route-edge passability state, Clearance interval guard...
M492–M499	Verification and ledger	8	Proposal verifier, Confidence floor, Numeric error margin...
M500–M505	Change and merge	6	Stable-ID repeat-scan match, Movement uncertainty interval, State-change candidate...
M506–M509	Export and governance	4	Canonical spatial JSON, GeoJSON projection, Self-contained offline HTML report...

A.2 Complete 60-entry delta

ID	Domain	Mechanism	Definition and substrate integration	Validation
M450	Capture and calibration	Versioned capture profile	A capture profile binds device family, camera model, calibration digest, coordinate frame, time base, model versions, scale mode and privacy policy. Makes every observation interpretable under one declared acquisition contract.	Implemented tested
M451	Capture and calibration	Camera calibration content digest	Calibration identity is recorded as a content digest or explicit unverified state rather than hidden application configuration. Enables repeatable association of measurements with the calibration that produced them.	Implemented tested
M452	Capture and calibration	Coordinate-frame declaration	Handedness, up axis and floor-plane convention are explicit project data. Prevents silent axis, handedness and unit changes between capture, map and export.	Implemented tested
M453	Capture and calibration	Metric scale mode	Scale is typed as metric_calibrated, anchored, relative or unknown. Measurement and route-clearance events can require metric-ready evidence.	Implemented tested
M454	Capture and calibration	Scale-anchor identity	Anchored scale requires a stable anchor ID and declared units-per-meter. Separates a real-world scale witness from a model guess.	Implemented tested
M455	Capture and calibration	Frame and timestamp identity	Every observation carries a frame ID and finite timestamp under a declared unit. Supports deterministic ordering, replay and source traceability.	Implemented tested
M456	Capture and calibration	Model-version provenance	Depth, topology, semantics and uncertainty model versions are part of the capture profile. Makes model upgrades and mixed-source evidence auditable.	Implemented tested
M457	Capture and calibration	Privacy/export policy	Capture records declare local-only, redacted-export, consented-sync or unrestricted handling. Provides an explicit governance gate before evidence leaves the device.	Schema + run guard tested
M458	Observation and uncertainty	Typed observation proposal	A proposal records stable ID, profile, frame, time, kind, pose, uncertainty, confidence, support, compatibility, guard status and provenance. Transforms model output into a finite inspectable candidate rather than a direct map write.	Implemented tested
M459	Observation and uncertainty	Normalized pose record	Position is finite and orientation is a normalized quaternion. Provides a stable 3D transform convention across evidence and map nodes.	Implemented tested
M460	Observation and uncertainty	Three-axis uncertainty record	Observation uncertainty stores non-negative axis sigmas, confidence level and optional conservative maximum error. Keeps measurement confidence separate from semantic confidence.	Implemented tested

M450–M509 catalog continued

ID	Domain	Mechanism	Definition and substrate integration	Validation
M461	Observation and uncertainty	Conservative position bound	A declared maximum error is authoritative; otherwise the reference bound is three times the Euclidean sigma norm. Feeds support expansion, verification limits and change intervals.	Implemented tested
M462	Observation and uncertainty	Relation residual interval	A scalar relation residual is enclosed by its numerical error. Prevents one floating value from hiding uncertainty around a guard.	Implemented tested
M463	Observation and uncertainty	Guard interval classification	Residual intervals are classified as inside, outside, guard, crossing or ambiguous under a declared margin. Separates robust relation classes from threshold-adjacent uncertainty.	Implemented tested
M464	Observation and uncertainty	Semantic label map	Typed semantics are carried as explicit key/value metadata and never replace stable identity. Allows doorway, obstacle, room and route meaning to participate in compatibility.	Implemented tested
M465	Observation and uncertainty	Static/movable/dynamic state	Motion class is a separate field with static, movable, dynamic or unknown values. Prevents moving people or objects from silently becoming permanent map geometry.	Implemented tested
M466	Observation and uncertainty	Evidence reference URI	An observation may retain a local or external evidence URI without embedding raw media in the hot map record. Supports review and low-bandwidth maps while preserving evidence linkage.	Implemented tested
M467	Observation and uncertainty	Deterministic observation ordering	Observation proposals sort by event time, descending priority, source and stable proposal ID. Makes batch ingestion reproducible regardless of detector arrival order.	Implemented tested
M468	Spatial support and indexing	Spherical support volume	A sphere admits points within radius plus a declared uncertainty expansion. Provides local neighborhood support for landmarks and repeated observations.	Implemented tested
M469	Spatial support and indexing	Axis-aligned box support	A finite AABB admits points inside bounds expanded by uncertainty. Provides simple room, scan-cell and site support.	Implemented tested
M470	Spatial support and indexing	Finite cone-frustum support	Apex, unit axis, near/far range and half-angle cosine define a bounded camera-like support. Adapts the retained cone-support calculus to phone view frusta.	Implemented tested
M471	Spatial support and indexing	Support registry	Versioned support IDs resolve to one finite support record; duplicate and unknown IDs reject. Keeps support references stable across proposals and projects.	Implemented tested
M472	Spatial support and indexing	Semantic compatibility policy	Required/forbidden tags, allowed kinds, motion classes, floor and semantic equalities form a typed compatibility gate. Ensures co-location is not sufficient for map coupling.	Implemented tested
M473	Spatial support and indexing	Voxel20x3 level4 key	A 64-bit word stores a 4-bit level and three signed 20-bit voxel coordinates. Provides deterministic sparse world-cell keys with exact pack/unpack tests.	Implemented tested
M474	Spatial support and indexing	Ray log-polar 20/18/14/12 key	A 64-bit camera-ray key packs log depth, azimuth, elevation and modular time. Reuses SCLP log-polar packing ideas for finite camera-ray indexing.	Implemented tested
M475	Spatial support and indexing	Deterministic spatial hash index	Items are bucketed by voxel key and sphere queries return distance/ID ordered results. Supplies a conventional index before semantic support and guard verification.	Implemented tested
M476	Map and identity	Stable map-node identity	A node ID remains stable across pose, semantic, state and evidence revisions. Separates logical object identity from coordinates and derived geometry.	Implemented tested
M477	Map and identity	Node revision lineage	Accepted node changes increment revision and append evidence and lineage labels. Makes repeat-scan updates traceable without replacing the node ID.	Implemented tested
M478	Map and identity	Stable map-edge identity	Edges have stable IDs, typed endpoints, route/topology kind, state, metrics, evidence and lineage. Represents portals, routes and relations independently of node coordinates.	Implemented tested
M479	Map and identity	Canonical map-state hash	Sorted canonical JSON over metadata, nodes and edges produces a SHA-256 state identity. Supports checkpoint integrity, replay and delta base negotiation.	Implemented tested
M480	Map and identity	Typed map patch family	Upsert, remove, state, pose, metric and evidence patches are explicit operations. Prevents hidden mutation and gives every event a replayable state transition.	Implemented tested
M481	Map and identity	Evidence attachment union	Repeated evidence IDs are de-duplicated while prior evidence remains attached. Preserves support history across revisions.	Implemented tested
M482	Map and identity	Explicit unknown state	Unknown, blocked and passable states remain distinct rather than coercing absence into false certainty. Allows incomplete scans to remain honest and route policies to decide how to treat unknowns.	Implemented tested
M483	Map and identity	Derived-geometry authority boundary	Meshes, plan projections and GeoJSON are downstream views unless an application explicitly promotes them under an error contract. Carries forward the independent geometry/error boundary from KC Two Hands 3.0.	Specified + enfo by API shape
M484	Topology and route	Room/portal/waypoint node classes	Route topology uses explicit node kinds instead of inferring connectivity from proximity. Prevents overlapping coordinates or floors from becoming accidental routes.	Implemented tested
M485	Topology and route	Route-edge passability state	Each route edge records passable/open, blocked/closed, restricted or unknown status. Makes blockage a first-class state change.	Implemented tested
M486	Topology and route	Clearance interval guard	Declared clearance and error produce a conservative interval checked against policy minimum. Supports accessibility-oriented route filtering with uncertainty.	Implemented tested
M487	Topology and route	Slope interval guard	Absolute slope plus error is checked against a maximum. Prevents uncertain steep segments from being silently accepted.	Implemented tested
M488	Topology and route	Trinary route knowledge	Passable, blocked and unknown are not collapsed to a Boolean. Allows strict or exploratory route profiles.	Implemented tested
M489	Topology and route	Deterministic accessible path	Dijkstra search uses guarded edges and stable cost/signature tie breaking. Returns reproducible route paths and rejected-edge reasons.	Implemented tested
M490	Topology and route	Route lineage	Route-edge revisions append evidence and lineage labels independently from node identity. Supports before/after passability history.	Implemented tested

M450–M509 catalog continued

ID	Domain	Mechanism	Definition and substrate integration	Validation
M491	Topology and route	Route-topology validation	Route edges require existing endpoints, positive length, recognized status and non-isolated portal participation. Rejects malformed route graphs before field use.	Implemented tested
M492	Verification and ledger	Proposal verifier	A proposal is accepted only after capture profile, support, compatibility, guard, confidence, uncertainty, scale and provenance checks. Implements the canonical UGTS authority chain for spatial observations.	Implemented tested
M493	Verification and ledger	Confidence floor	Observation confidence must meet a versioned policy floor. Prevents low-confidence model outputs from directly mutating the map.	Implemented tested
M494	Verification and ledger	Numeric error margin	Observation error must remain below both policy and event margins. Keeps finite-precision uncertainty subordinate to event classification.	Implemented tested
M495	Verification and ledger	Metric-scale requirement	Doorway, route and measurement kinds can require calibrated or anchored scale. Blocks false metric precision from relative-depth captures.	Implemented tested
M496	Verification and ledger	Deterministic authoritative commit	Verified proposals commit in stable event-time/priority/source/ID order. Separates parallel detection from one authoritative mutation boundary.	Implemented tested
M497	Verification and ledger	Patch conflict key	Same-time patches to the same typed target field conflict; first deterministic winner commits and the rest receive reasons. Makes merge/rejection behavior inspectable.	Implemented tested
M498	Verification and ledger	Pre/post state hash chain	Every committed event stores the map hash before and after its patch. Supports replay divergence checks and evidence-chain inspection.	Implemented tested
M499	Verification and ledger	Checkpoint and replay	A checkpoint stores map state, sequence, state hash and event-stream hash; replay stops at the first mismatch. Provides deterministic reconstruction and bounded failure reporting.	Implemented tested
M500	Change and merge	Stable-ID repeat-scan match	Baseline and current nodes/edges are paired by stable ID before comparing coordinates or state. Avoids nearest-point ambiguity when logical identity is available.	Implemented tested
M501	Change and merge	Movement uncertainty interval	Displacement is enclosed by the sum of baseline and current position bounds. Verifies movement only when the entire interval exceeds the threshold.	Implemented tested
M502	Change and merge	State-change candidate	Selected state fields produce an explicit before/after candidate. Captures passability and presence changes separately from geometry.	Implemented tested
M503	Change and merge	Added/removed candidate	Stable IDs present in only one scan become added or removed candidates with confidence. Makes map completeness and deletions reviewable.	Implemented tested
M504	Change and merge	Content-addressed proposal delta	A low-bandwidth delta carries base map hash, source ID and sorted proposals plus a content hash. Supports exchange before dense media or meshes are transferred.	Implemented tested
M505	Change and merge	Deterministic multi-source delta merge	Deltas sharing a base hash are de-duplicated, sorted and checked for same-time patch conflicts. Provides a reference collaborative-map merge without unchecked state writes.	Implemented tested
M506	Export and governance	Canonical spatial JSON	Projects, ledgers, events and deltas serialize as canonical versioned JSON with stable hashes. Provides the authoritative interchange and persistence form.	Implemented tested
M507	Export and governance	GeoJSON projection	Nodes become points and edges become line strings while UGTS state, uncertainty and lineage remain properties. Offers conventional GIS inspection as a downstream projection.	Implemented tested
M508	Export and governance	Self-contained offline HTML report	A one-file HTML report embeds map, routes, changes, event table, hashes and JSON with no network asset. Supports field transfer and inspection without an app store or server.	Implemented tested
M509	Export and governance	Android implementation deferral contract	Version 4.0 freezes the attached 3.9.1 NativeActivity/GLES3 implementation as reference and introduces no phone-specific app changes. Ensures substrate interfaces are completed before later POCO-specific native capture/viewer work.	Specified + erence tree h audited

B. Release identifiers and attribution

Runtime version	4.0.0
Edition	Spatial Evidence Ledger
Project schema	`ugts-kc-spatial-project-4.0`
Mechanism range	M001–M509 by retained source/catalog
Automated tests	380
Android implementation status	Deferred; 3.9.1 source reference frozen unchanged
Requester attribution	Tom Klootwijk; NL200678942; 10 July 1990 - supplied by requester, not independently verified

EVIDENCE BOUNDARY

This report and package are not legal proof of identity, authorship, ownership, patentability, priority, exclusive rights, licensing status or chain of title. Hashes provide integrity and divergence checks within the declared package; they are not digital signatures or independent provenance certification.